Worksheet — 2.2 Problem Solving & Programming — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

1. Key-term match: 1 → C; 2 → E; 3 → B; 4 → A; 5 → H; 6 → D; 7 → F; 8 → G.

2. Scope (worked): - a) **10** — the global `score`, before any subroutine runs. - b) **5** — the local `score` inside `tryLocal()`. - c) **99** — `addPoints()` assigned to the **global** `score`, so the change persists. - d) The `score` inside `tryLocal()` is a **local** variable that **shadows** the global only within that subroutine; it is destroyed when the subroutine ends, so it never affects the global value the final `print` reads.

3. By value vs by reference (worked): - a) `a` prints **1** — `increase` received a **copy** (by value), so the original was unchanged. - b) `b` prints **101** — `boost` received the **address** (by reference), so adding 100 changed the original. - c) The consequence: with **by value** the original variable is **unchanged** because only a copy is altered; with **by reference** the original **can be (and is) changed** because the subroutine works on the same memory location.

4. Recursion trace:

Call (frame pushed)	Base case?	Expression returned	Value returned (on pop)
factorial(4)	No	4 * factorial(3)	4 * 6 = **24**
factorial(3)	No	3 * factorial(2)	3 * 2 = **6**
factorial(2)	No	2 * factorial(1)	2 * 1 = **2**
factorial(1)	No	1 * factorial(0)	1 * 1 = **1**
factorial(0)	**Yes**	return 1	**1**

Frames are **pushed** down to `factorial(0)`, then **popped** back up in LIFO order, each multiplying its `n` by the value returned from below.

- (b) Final value printed: **24**.
- (c) Without the base case, `factorial(0)` would call `factorial(-1)`, then `factorial(-2)`, … with **no stopping condition**. Each call **pushes another stack frame**; the call stack fills up and a **stack overflow** error occurs.

5. Class (model answer):

```
class Vehicle
    private make
    private numWheels

    public procedure new(m, w)
        make = m
        numWheels = w
    endprocedure

    public procedure describe()
        print("Make: " + make + ", wheels: " + numWheels)
    endprocedure
endclass

class Car inherits Vehicle
    private doors

    public procedure new(m, w, d)
        super.new(m, w)
        doors = d
    endprocedure

    public procedure describe()
        super.describe()
        print("Doors: " + doors)
    endprocedure
endclass

myCar = new Car("Ford", 4, 5)
myCar.describe()
```

Output:

```
Make: Ford, wheels: 4
Doors: 5
```

- b) **Encapsulation** (data hiding — private attributes accessed only via methods).
- c) **Polymorphism** via **overriding** (the subclass replaces the inherited method with the same name and parameters).

6. Computational methods: a) **Pipelining**; b) **Data mining**; c) **Heuristics**; d) **Backtracking**; e) **Performance modelling**; f) **Visualisation**.

7. Challenge (model answer):

```
procedure countdownR(n)
    if n == 0 then              // BASE CASE
        print("Liftoff")
    else                        // RECURSIVE CASE
        print(n)
        countdownR(n - 1)
    endif
endprocedure
```

- (ii) **Advantage** (any one): shorter/more elegant code that mirrors the self-similar structure; no explicit loop variable to manage. **Disadvantage** (any one): uses more memory (a stack frame per call); slower due to call overhead; risk of **stack overflow** for large `n`.

8. Recursion trace II (worked):

Working: `4726 MOD 10 = 6`, `4726 DIV 10 = 472`; `472 MOD 10 = 2`, `472 DIV 10 = 47`; `47 MOD 10 = 7`, `47 DIV 10 = 4`; `4 < 10` → base case.

Call (frame pushed)	Base case reached?	Expression returned	Value returned (on pop)
sumDigits(4726)	No	6 + sumDigits(472)	6 + 13 = **19**
sumDigits(**472**)	No	**2** + sumDigits(**47**)	2 + 11 = **13**
sumDigits(**47**)	No	**7** + sumDigits(**4**)	7 + 4 = **11**
sumDigits(**4**)	**Yes**	return **4**	**4**

- (b) Final value printed: **19** (check: 4 + 7 + 2 + 6 = 19 ✓).
- (c) Maximum frames on the stack at once: **4** — all four calls are active when `sumDigits(4)` is reached, before any frame has been popped.
- (d) `sumDigits(4)` returns first. It was the **last frame pushed**, and the stack is **LIFO** (last in, first out): no earlier call can finish because each is **waiting for the result** of the call it made, so frames must be popped in the reverse of the order they were pushed.

9. Find and fix (worked):

(a)

Line	Syntax or logic?	What is wrong	Corrected line / fix
02	**Logic**	Initialising `max = 0` fails when **all values are negative** — 0 is returned even though 0 is not in the array.	`max = arr[0]`
03	**Logic** (causes a run-time out-of-range error)	The loop runs to `arr.length`, but the highest valid index is `arr.length - 1` — `arr[arr.length]` is out of bounds.	`for i = 0 to arr.length - 1` (or `for i = 1 to arr.length - 1` once `max = arr[0]`)
04–06	**Syntax**	The `if` statement has no matching `endif` before `next i`.	Insert `endif` after line 05

(b) Corrected function:

```
function findMax(arr:array)
    max = arr[0]
    for i = 1 to arr.length - 1
        if arr[i] > max then
            max = arr[i]
        endif
    next i
    return max
endfunction
```

(Starting the loop at `i = 0` is also correct — comparing `arr[0]` with itself is harmless.)

Check with `arr = [-5, -2, -9]`: `max = -5`; `i = 1`: `-2 > -5` → `max = -2`; `i = 2`: `-9 > -2` false. Returns **-2** ✓.

(c) Test data: an array of **all negative values**, e.g. `[-3, -7, -1]`. Expected result: **-1** (the largest element). Actual result with `max = 0`: **0**, which is not even in the array — revealing the logic error.

(d) Only the **missing** `endif` (syntax error) is reported by the translator before the program runs, because it breaks the rules of the language and the code cannot be parsed. The other two are **logic errors**: the code is syntactically valid and runs, but produces wrong behaviour — these are only exposed by **running the program with well-chosen test data** (e.g. boundary/erroneous data such as an all-negative array).

Section B — model answers

Q1 (2 marks). One mark per bullet: - A **local** variable is declared inside a subroutine, is only accessible (in scope) within that subroutine, and exists only while the subroutine is running (created on call, destroyed on return). **[1]** - A **global** variable is declared outside all subroutines, is accessible throughout the whole program, and exists for the entire run of the program. **[1]**

Q2 (3 marks). - (i) Passing **by value** would create a **complete copy** of the large array **[1]**, which wastes memory and takes time; passing **by reference** passes only the **memory address**, so no copy is made regardless of the array's size **[1]**. - (ii) The subroutine can **modify the original array** — an unintended change (side effect) affects the rest of the program. **[1]**

Q3 (4 marks).

(i) Trace table — one mark per correct row **[3]**:

a	b	r	Output
5	6	0	
10	3	0	
20	1	10	
40	0	30	30

Working: iteration 1 — `6 MOD 2 = 0` so `r` unchanged; `a = 10`, `b = 3`. Iteration 2 — `3 MOD 2 = 1` so `r = 0 + 10 = 10`; `a = 20`, `b = 1`. Iteration 3 — `1 MOD 2 = 1` so `r = 10 + 20 = 30`; `a = 40`, `b = 0`. Loop ends (`b = 0`); function returns **30**.

(ii) It **multiplies the two parameters** — it returns `a × b` (here 5 × 6 = 30). **[1]** (This is "Russian peasant" multiplication by doubling and halving.)

Q4 (2 marks). Any two, one mark each: - **Breakpoints** — pause execution at a chosen line to inspect the program state. - **Stepping** (step over / step into) — execute one line at a time to follow the flow. - **Watch window / variable inspection** — view the current values of variables as the code runs. - **Trace / debug output** facilities showing the order in which statements execute.

Q5 (5 marks). Model answer:

```
function countMatches(scores:array, target)
    count = 0
    for i = 0 to scores.length - 1
        if scores[i] == target then
            count = count + 1
        endif
    next i
    return count
endfunction
```

Mark points (one each): - Function declaration with **both parameters**, ended with `endfunction`. **[1]** - Count variable **initialised to 0** before the loop. **[1]** - Loop that visits **every element** for an array of any length (`0` to `scores.length - 1`). **[1]** - Correct **comparison** of the current element with `target` inside the loop. **[1]** - Count **incremented** on a match and **returned** after the loop. **[1]**

Q6 (3 marks). - Create a **superclass** (e.g. `Character`) containing the **shared** attributes `name` and `health` and the shared method `move()`. **[1]** - Declare `Wizard` and `Warrior` as **subclasses** (`class Wizard inherits Character`), so they automatically gain the shared attributes/methods **without duplicating code**; constructors call `super.new(...)`. **[1]** - Each subclass then **adds its own members** (e.g. `castSpell()` in `Wizard` only) and/or **overrides** inherited methods where behaviour differs; changes to shared code are made once in `Character`. **[1]**

Q7 (2 marks). - The number of possible routes grows **combinatorially/exponentially** with the number of junctions, so examining every route is **computationally infeasible** in an acceptable time. **[1]** - A heuristic is a **rule of thumb** (e.g. straight-line distance to the destination) that guides the search towards **promising** junctions only, producing a **good-enough route quickly**, trading guaranteed optimality for speed. **[1]**

Q8* (9 marks) — levels of response.

- **Level 3 (7–9 marks):** Thorough knowledge and understanding of OOP. All four required features (classes, inheritance, encapsulation, polymorphism) are accurate and **applied to the vet scenario** (e.g. names the classes and attributes). There is a well-developed line of reasoning which is clear and logically structured; benefits are substantiated and a reasoned conclusion is given.
- **Level 2 (4–6 marks):** Reasonable knowledge and understanding. Most features described correctly with some application to the scenario; some development of points. The line of reasoning has some structure; occasional inaccuracies or generic (unapplied) statements.
- **Level 1 (1–3 marks):** Basic knowledge only. Isolated or generic points about OOP with little or no application to the scenario; little evidence of a line of reasoning.
- **0 marks:** No response worthy of credit.

Model top-band answer:

The developer should create an `Animal` **class** as a template holding everything common to all animals: **private attributes** `name`, `owner` and `dateOfLastVisit`, a constructor `new(...)` to set them, and public methods such as `describe()` and getters/setters. Making the attributes private is **encapsulation**: the data can only be read or changed through the class's methods, so the class can **validate** changes (e.g. reject a last-visit date in the future) and other parts of the program cannot corrupt the data. This reduces bugs and means the internal representation can change without breaking other code.

`Dog`, `Cat` and `Rabbit` should be **subclasses** that **inherit** from `Animal` (`class Dog inherits Animal`). Each constructor calls `super.new(name, owner, dateOfLastVisit)` and then sets its own private attribute — `microchipped` for `Dog`, `indoorOnly` for `Cat`; `Rabbit` needs no extra attributes. Inheritance means the common code is written **once**, avoiding duplication: adding a new common field (e.g. owner's phone number) to `Animal` automatically applies to all three species, making the system easier to maintain and extend (a new `Bird` class later is just one more subclass).

Polymorphism lets each subclass **override** `describe()` — `Dog.describe()` calls `super.describe()` then prints its microchip status. The surgery's records can then be held in a single array of `Animal` objects and processed with one loop:

```
class Animal
    private name
    private owner
    private dateOfLastVisit
    public procedure new(n, o, d)
        name = n
        owner = o
        dateOfLastVisit = d
    endprocedure
    public procedure describe()
        print(name + ", owner: " + owner + ", last visit: " + dateOfLastVisit)
    endprocedure
endclass

class Dog inherits Animal
    private microchipped
    public procedure new(n, o, d, chip)
        super.new(n, o, d)
        microchipped = chip
    endprocedure
    public procedure describe()
        super.describe()
        print("Microchipped: " + microchipped)
    endprocedure
endclass
```

Calling `animals[i].describe()` runs the **correct version for whichever subclass** each object belongs to, without the calling code needing to check the type.

Conclusion: OOP suits this system well because the data is naturally hierarchical (species sharing common animal data). Classes give reusable templates, inheritance removes duplication, encapsulation protects the surgery's records, and polymorphism keeps the record-processing code simple — producing a design that is easier to maintain and extend than a procedural equivalent, at the minor cost of some initial design effort.